

tablica rekord lista stos slownik

Struktury danych

Dział II · Projektowanie struktur danych

Projektowanie oprogramowania

Technik programista · klasa II · INF.04.3.2

Opracował: Bartosz Bryniarski

Plan działu

Sześć lekcji

1 Tablice jednowymiarowe i dwuwymiarowe

Indeksowanie, układ w pamięci

2 Tablice dynamiczne i asocjacyjne

Realokacja, klucz i wartość

3 Typ rekordowy: struktura i unia

Dane niejednorodne w jednym obiekcie

4 Typ plikowy i typ wskaźnikowy

Trwałość danych, adres w pamięci

5 Kolekcje: listy, kolejki, stosy, wektory

Znaczenie iteratora

6 Projektowanie zestawu danych

Dobór struktury do problemu

1

Tablice jednowymiarowe i dwuwymiarowe

Podstawowa struktura danych

indeksowanie · układ w pamięci · macierz · złożoność

Po co nam tablica

Sto zmiennych albo jedna tablica

Bez tablicy

```
int ocena1 = 5;
int ocena2 = 4;
int ocena3 = 3;
...
// jak policzyc srednie?
// jak dodac kolejna ocene?
```

Z tablicą

```
int oceny[30];

for (i = 0; i < 30; i++)
    suma += oceny[i];

// jedna petla, dowolny rozmiar
```

Tablica to zbiór elementów tego samego typu, umieszczonych w pamięci jeden za drugim i dostępnych przez indeks liczbowy.

Tablica w pamięci

Dlaczego dostęp jest natychmiastowy

```
int tab[5] = {10, 20, 30, 40, 50};    // każdy int = 4 bajty
```

indeks:	0	1	2	3	4
wartosc:	10	20	30	40	50
adres:	1000	1004	1008	1012	1016

```
adres elementu = adres poczatku + indeks * rozmiar typu
```

Jeden wzór zamiast szukania. Komputer nie przegląda tablicy, tylko oblicza adres. Dlatego odczyt dowolnego elementu trwa tyle samo, niezależnie od rozmiaru tablicy.

Deklaracja tablicy

Składnia w kilku językach

```
int liczby[5];           // C/C++ - rozmiar stały
int[] liczby = new int[5]; // Java
var liczby = new int[5];  // C#
liczby: array[0..4] of integer; // Pascal

liczby = [10, 20, 30, 40, 50] # Python - lista
const liczby = [10, 20, 30];  // JavaScript
```

Tablica statyczna

Rozmiar ustalany przy deklaracji i niezmienny. C, Java, Pascal.

Lista Pythona

Formalnie tablica dynamiczna - rozmiar rośnie w trakcie działania.

Indeksowanie i zakres

Najczęstszy błąd początkującego

```
int tab[5];           // elementy o indeksach 0, 1, 2, 3, 4

tab[0]    -> pierwszy element
tab[4]    -> ostatni element
tab[5]    -> POZA ZAKRESEM

for (i = 0; i <= 5; i++)    // bład: warunek powinien być i < 5
```

Java, C#, Python

Zgłaszają wyjątek - program zatrzymuje się w miejscu błędu

C i C++

Nie sprawdzają zakresu - program czyta cudzą pamięć i działa dalej

Błąd o jeden (ang. off-by-one) to klasyka - ostatni indeks to zawsze rozmiar minus jeden.

Typowe operacje na tablicy

Schematy, które trzeba znać

```
# suma elementow
suma = 0
for i in range(n):
    suma += tab[i]

# wartosc najwieksza
maks = tab[0]
for i in range(1, n):
    if tab[i] > maks:
        maks = tab[i]

# wyszukiwanie liniowe
for i in range(n):
    if tab[i] == szukana:
        return i
return -1

# odwracanie kolejnosci
for i in range(n // 2):
    tab[i], tab[n-1-i] = tab[n-1-i], tab[i]
```

Wartość początkową maksimum bierzemy z pierwszego elementu, nie z zera - inaczej algorytm zawiedzie dla samych liczb ujemnych.

Tablica dwuwymiarowa

Wiersze i kolumny

```
int macierz[3][4];          // 3 wiersze, 4 kolumny

      kol 0   kol 1   kol 2   kol 3
wiersz 0:   1     2     3     4
wiersz 1:   5     6     7     8
wiersz 2:   9    10    11    12

macierz[1][2] -> 7          // [wiersz][kolumna]
```

Obraz cyfrowy

Piksele jako siatka

Plansza gry

Szachownica, saper

Tabela danych

Wiersze i kolumny arkusza

Układ tablicy 2D w pamięci

Pamięć jest jednowymiarowa

```
macierz 2 x 3:  [1, 2, 3]
                  [4, 5, 6]
```

```
wierszami (row-major, C/C++/Java/Python):
```

```
  1  2  3  4  5  6
```

```
kolumnami (column-major, Fortran, MATLAB):
```

```
  1  4  2  5  3  6
```

Dlaczego to ważne: przeglądanie tablicy w kolejności zgodnej z układem w pamięci bywa kilkakrotnie szybsze, bo procesor wczytuje sąsiednie komórki jednym ruchem.

Przeglądanie tablicy 2D

Pętla w pętli

```
for (int w = 0; w < LICZBA_WIERSZY; w++) {  
    for (int k = 0; k < LICZBA_KOLUMN; k++) {  
        suma += macierz[w][k];  
    }  
}
```

```
# Python - iteracja po wierszach  
for wiersz in macierz:  
    for wartosc in wiersz:  
        suma += wartosc
```

Pętla zewnętrzna zawsze odpowiada pierwszemu indeksowi. Zamiana pętli miejscami daje inny porządek odczytu.

Złożoność operacji na tablicy

Ile to kosztuje

Operacja	Złożoność	Dlaczego
Odczyt elementu o indeksie i	$O(1)$	Adres liczony wzorem, bez przeglądania
Zmiana elementu o indeksie i	$O(1)$	Jak wyżej
Wyszukiwanie wartości	$O(n)$	W najgorszym razie trzeba sprawdzić wszystko
Wyszukiwanie w tablicy posortowanej	$O(\log n)$	Wyszukiwanie binarne - połowienie zakresu
Wstawienie w środek	$O(n)$	Trzeba przesunąć wszystkie dalsze elementy
Usunięcie ze środka	$O(n)$	Trzeba zasypać powstałą lukę

Zapis $O(n)$ opisuje, jak rośnie czas działania wraz ze wzrostem liczby danych.

Temat 1 – podsumowanie

Co trzeba umieć

- Tablica przechowuje elementy jednego typu w ciągłym obszarze pamięci
- Indeksowanie zaczyna się od zera, ostatni indeks to rozmiar minus jeden
- Adres elementu liczy się wzorem, dlatego odczyt po indeksie ma koszt stały
- Tablica statyczna ma rozmiar ustalony przy deklaracji i niezmienny
- Tablica dwuwymiarowa to siatka wierszy i kolumn: `macierz[wiersz][kolumna]`
- Dane 2D leżą w pamięci wierszami - stąd podwójna pętla z wierszem na zewnątrz
- Wstawianie i usuwanie w środku tablicy wymaga przesuwania elementów

2

Tablice dynamiczne i asocjacyjne

Gdy rozmiar nieznany, a indeks nie wystarcza

realokacja · koszt amortyzowany · klucz i wartość · funkcja skrótu

Ograniczenie tablicy statycznej

Rozmiar trzeba znać z góry

```
int uczniowie[30];      // a jesli klasa ma 31 osob?  
int uczniowie[1000];    // a jesli ma 12? marnujemy pamiec
```

Ile produktow bedzie w koszyku?
Ile wierszy ma plik, ktory wczytujemy?
Ilu uzytkownikow zaloguje sie dzisiaj?
Tego nie da sie ustalic podczas pisania programu.

Tablica dynamiczna sama powiększa swój rozmiar w trakcie działania programu. Programista dodaje elementy, a struktura dba o miejsce w pamięci.

Jak działa tablica dynamiczna

Powiększanie przez przepisanie

```
pojemnosc 4, zajete 4:  [A][B][C][D]
```

```
dodajemy element E:
```

1. rezerwujemy nowy, wiekszy obszar (zwykle dwa razy wiekszy)
2. kopiujemy dotychczasowe elementy
3. zwalniamy stary obszar

```
pojemnosc 8, zajete 5:  [A][B][C][D][E][ ][ ][ ][ ]
```

Rozmiar

Liczba faktycznie przechowywanych elementów

Pojemność

Liczba miejsc zarezerwowanych w pamięci - zwykle większa

Koszt dodawania elementu

Dlaczego podwajamy pojemność

dodawanie na koniec:

zwykle	-> wpisanie w wolne miejsce	szybkie
czasem	-> przepisanie całej tablicy	kosztowne

przy podwajaniu pojemności przepisanie zdarza się coraz rzadziej:
po 4, 8, 16, 32, 64 ... elementach

sredni koszt jednego dodania pozostaje staly - $O(1)$ zamortyzowane

Koszt zamortyzowany to koszt uśredniony po wielu operacjach. Pojedyncze dodanie bywa drogie, ale tysiąc dodań kosztuje tyle, co tysiąc tanich operacji.

Tablica dynamiczna w językach

Ta sama idea, różne nazwy

Język	Nazwa	Dodanie na koniec	Rozmiar
Python	list	<code>lista.append(x)</code>	<code>len(lista)</code>
C++	<code>std::vector</code>	<code>v.push_back(x)</code>	<code>v.size()</code>
Java	ArrayList	<code>lista.add(x)</code>	<code>lista.size()</code>
C#	<code>List<T></code>	<code>lista.Add(x)</code>	<code>lista.Count</code>
JavaScript	Array	<code>tab.push(x)</code>	<code>tab.length</code>
PHP	array	<code>\$tab[] = \$x;</code>	<code>count(\$tab)</code>

Nazwa wektor nie ma tu nic wspólnego z wektorem z matematyki - to po prostu tablica dynamiczna.

Tablica asocjacyjna

Gdy indeks liczbowy nie ma sensu

Jak przechować liczbę ludności miast?

```
miasta[0] = 1860000    # które miasto ma indeks 0?
```

```
miasta["Warszawa"] = 1860000
```

```
miasta["Krakow"]    = 804000
```

```
miasta["Wroclaw"]   = 674000
```

Tablica asocjacyjna przechowuje pary klucz - wartość. Zamiast liczbowego indeksu używamy klucza, którym najczęściej jest napis. Inne nazwy: słownik, mapa, tablica mieszająca.

Jak działa słownik

Funkcja skrótu i kubetki

```
klucz "Warszawa"  
    |  
    v   funkcja skrotu (hash)  
liczba 7391204  
    |  
    v   reszta z dzielenia przez liczbe kubelkow  
kubelek nr 4    ->    tu zapisujemy wartosc
```

Zaleta

Odczyt po kluczu ma koszt stały - nie przeglądamy całej struktury

Kolizja

Dwa klucze trafiają do tego samego kubetka - struktura radzi sobie z tym sama

Słownik nie gwarantuje kolejności zapisu jako kolejności przechowywania - w Pythonie od wersji 3.7 kolejność dodawania jest zachowana.

Słownik w praktyce

Składnia i podstawowe operacje

```
# Python
oceny = {}
oceny["Kowalski"] = 5
ocena = oceny["Kowalski"]
"Nowak" in oceny    -> False
del oceny["Kowalski"]
for k, v in oceny.items():

// Java
Map<String,Integer> oceny
    = new HashMap<>();
oceny.put("Kowalski", 5);
oceny.get("Kowalski");
oceny.containsKey("Nowak");
oceny.remove("Kowalski");
```

Przed odczytem sprawdzamy, czy klucz istnieje. Odwołanie do nieistniejącego klucza kończy się błędem w Pythonie, a wartością pustą w Javie i JavaScriptcie.

Wymagania wobec klucza

Nie wszystko nadaje się na klucz

Niezmienny

Klucz nie może się zmienić po dodaniu
- inaczej trafi do złego kubeczka i
zniknie

Jednoznaczny

Ten sam klucz nie może opisywać
dwóch różnych rzeczy

Porównywalny

Musi dać się policzyć jego skrót i
sprawdzić równość z innym kluczem

dobre klucze: "PL-2024-001" numer_pesel id_produktu

złe klucze: lista ocen obiekt zmieniany w trakcie

W Pythonie kluczem może być napis, liczba lub krotka, ale nie lista - bo listę da się zmienić.

Tablica czy słownik

Jak wybrać

Kryterium	Tablica / lista	Słownik
Dostęp do elementu	Po indeksie liczbowym	Po kluczu dowolnego typu
Kolejność	Istotna i zachowana	Drugorzędna
Wyszukiwanie wartości	$O(n)$ - przeglądanie	$O(1)$ - po kluczu
Typowe zastosowanie	Lista ocen, kolejka zgłoszeń	Słownik pojęć, konfiguracja, licznik
Zużycie pamięci	Mniejsze	Większe - trzeba trzymać klucze

Pytanie kontrolne: czy odwołuję się do danych przez pozycję, czy przez nazwę? Pozycja prowadzi do tablicy, nazwa do słownika.

Temat 2 – podsumowanie

Co trzeba umieć

- Tablica dynamiczna sama zwiększa rozmiar - rezerwuje większy obszar i przepisuje dane
- Rozmiar to liczba elementów, pojemność to liczba zarezerwowanych miejsc
- Dodawanie na koniec ma stały koszt amortyzowany dzięki podwajaniu pojemności
- Nazwy w językach: list, vector, ArrayList, List<T>, Array
- Tablica asocjacyjna przechowuje pary klucz - wartość; inne nazwy: słownik, mapa
- Klucz zamieniany jest funkcją skrótu na numer kubłka, stąd stały koszt odczytu
- Klucz musi być niezmienny i jednoznaczny - lista nie nadaje się na klucz

3

Typ rekordowy: struktura i unia

Dane niejednorodne opisujące jeden obiekt

rekord · pola · tablica rekordów · unia

Problem danych rozproszonych

Trzy tablice opisujące te same osoby

```
// złe - dane jednego ucznia leża w trzech miejscach
string imiona[30];
int wieki[30];
float srednie[30];

// usuwamy ucznia numer 7 - trzeba pamiętać o wszystkich trzech tablicach
// zamiana kolejności w jednej tablicy rozspaja dane
```

Typ rekordowy łączy dane różnych typów, które razem opisują jeden obiekt, w jedną całość o własnej nazwie. Inne nazwy: rekord, struktura, krotka nazwana.

Struktura w C i C++

Definicja i użycie

```
struct Uczeń {  
    char imie[30];        // pole typu znakowego  
    int  wiek;            // pole typu całkowitego  
    float srednia;        // pole zmiennoprzecinkowe  
};  
  
struct Uczeń u;  
u.wiek = 17;              // dostęp przez kropkę  
printf("%d", u.wiek);
```

Definicja typu

Opisuje wzorzec - nie rezerwuje jeszcze pamięci

Zmienna typu

Konkretny obiekt z własnymi wartościami pól

Rekord w innych językach

Ta sama idea, inna składnia

```
# Python - dataclass
@dataclass
class Uczeń:
    imie: str
    wiek: int
    srednia: float

u = Uczeń("Ala", 17, 4.5)
print(u.wiek)
```

```
// Java - record
record Uczeń(
    String imie,
    int wiek,
    double srednia) {}

// JavaScript - obiekt
const u = { imie: "Ala",
            wiek: 17 };
```

W językach obiektowych rolę rekordu pełni klasa bez metod. Rekord przechowuje dane, klasa dodaje do nich zachowania.

Rekord w pamięci

Pola leżą jedno po drugim

```
struct Punkt {  
    int x;          // 4 bajty  
    int y;          // 4 bajty  
};                 // razem 8 bajtów  
  
struct Mieszane {  
    char znak;      // 1 bajt  
                  // 3 bajty wypełnienia (padding)  
    int liczba;     // 4 bajty  
};                 // razem 8 bajtów, nie 5
```

Procesor czyta dane szybciej, gdy zaczynają się pod adresem podzielnym przez ich rozmiar - stąd wypełnienie.

Tablica rekordów

Podstawowy zestaw danych aplikacji

```
struct Uczeń klasa[30];

klasa[0].imie      = "Ala";
klasa[0].srednia   = 4.75;

for (int i = 0; i < 30; i++)
    if (klasa[i].srednia > 4.5)
        printf("%s", klasa[i].imie);
```

To jest wzorzec, na którym opiera się większość aplikacji: rekord odpowiada wierszowi tabeli w bazie danych, a tablica rekordów - całej tabeli.

Rekordy zagnieżdżone

Rekord jako pole innego rekordu

```
struct Adres {  
    char ulica[50];  
    char miasto[30];  
    char kod[7];  
};  
  
struct Klient {  
    char nazwa[60];  
    struct Adres adres;           // rekord w rekordzie  
};  
  
k.adres.miasto           // dostep przez kolejne kropki
```

Zagnieżdżanie pozwala opisać dane złożone bez tworzenia dziesiątek osobnych pól.

Unia

Jedno miejsce w pamięci, kilka interpretacji

```
union Wartosc {  
    int    calkowita;    // 4 bajty  
    float  rzeczywista;  // 4 bajty  
    char   znaki[4];     // 4 bajty  
}; // rozmiar unii = 4 bajty, nie 12  
  
union Wartosc w;  
w.calkowita = 65;  
w.rzeczywista = 1.5; // nadpisuje poprzednia wartosc
```

Wszystkie pola unii zajmują ten sam obszar pamięci. W danej chwili sensowną wartość ma tylko jedno z nich.

Struktura a unia

Zestawienie

Cecha	Struktura (rekord)	Unia
Pamięć	Każde pole ma własne miejsce	Wszystkie pola dzielą jedno miejsce
Rozmiar	Suma rozmiarów pól (plus wypełnienie)	Rozmiar największego pola
Aktywne pola	Wszystkie jednocześnie	Jedno w danej chwili
Zastosowanie	Opis obiektu o wielu cechach	Oszczędność pamięci, odczyt tych samych bajtów na kilka sposobów

Unia z polem rozpoznawczym. W praktyce unię łączy się ze strukturą zawierającą dodatkowe pole, które mówi, które pole unii jest aktualnie użyte.

Unia z polem rozpoznawczym

Bezpieczny sposób użycia

```
enum Rodzaj { LICZBA, TEKST };

struct Dana {
    enum Rodzaj rodzaj;           // ktore pole jest uzyte
    union {
        int    liczba;
        char   tekst[20];
    } wartosc;
};

if (d.rodzaj == LICZBA) printf("%d", d.wartosc.liczba);
```

Bez pola rozpoznawczego program nie ma jak sprawdzić, jak odczytać zawartość unii.

Temat 3 – podsumowanie

Co trzeba umieć

- Typ rekordowy łączy dane różnych typów opisujące jeden obiekt
- Pola rekordu mają nazwy; odwołujemy się do nich przez kropkę
- Definicja typu to wzorzec - pamięć rezerwuje dopiero zmienna tego typu
- Rozmiar struktury to suma rozmiarów pól powiększona o wypełnienie
- Tablica rekordów odpowiada tabeli w bazie danych: rekord to wiersz, pole to kolumna
- W unii wszystkie pola zajmują ten sam obszar - rozmiar równy największemu polu
- Unii używa się z polem rozpoznawczym, które mówi, które pole jest aktywne

4

Typ plikowy i typ wskaźnikowy

Trwałość danych i adresy w pamięci

plik tekstowy i binarny · tryby otwarcia · adres · stos i sarta

Po co typ plikowy

Pamięć operacyjna jest ulotna

Pamięć operacyjna

Szybka, ale znika po zamknięciu programu. Tu żyją tablice i rekordy.

Plik na dysku

Wolniejszy, lecz trwały. Dane przetrwają wyłączenie komputera.

```
typ plikowy = zmienna reprezentujaca otwarty plik  
  
program <---> uchwyt pliku <---> dane na dysku
```

Każdy program, który ma zapamiętać cokolwiek między uruchomieniami, korzysta z pliku albo z bazy danych.

Plik tekstowy i binarny

Dwa sposoby zapisu

Cecha	Plik tekstowy	Plik binarny
Zawartość	Znaki czytelne dla człowieka	Bajty w postaci wewnętrznej programu
Liczba 1000	Cztery znaki: 1 0 0 0	Cztery bajty wartości typu int
Odczyt w notatniku	Czytelny	Nieczytelny
Rozmiar	Zwykle większy	Zwykle mniejszy
Przykłady	txt, csv, json, html	jpg, mp3, exe, dat

Plik tekstowy wymaga ustalenia kodowania znaków - temat z działu I wraca tutaj.

Wymiana danych między systemami idzie zwykle plikiem tekstowym, bo jest niezależny od platformy.

Operacje na pliku

Zawsze w tej samej kolejności

```
1. otworz    ->    2. czytaj albo zapisuj    ->    3. zamknij
```

tryby otwarcia:

"r"	odczyt	- plik musi istniec
"w"	zapis	- kasuje dotychczasowa zawartosc
"a"	dopisywanie	- dodaje na koncu pliku
"r+"	odczyt i zapis	
"rb" / "wb"		- wariant binarny

Tryb zapisu kasuje zawartość pliku bez ostrzeżenia. Do dopisywania danych służy tryb dopisywania, nie tryb zapisu.

Praca z plikiem w kodzie

Python i C

```
# Python - konstrukcja with zamyka plik automatycznie
with open("dane.txt", "r", encoding="utf-8") as f:
    for linia in f:
        print(linia.strip())

/* C */
FILE *f = fopen("dane.txt", "r");
if (f == NULL) { /* obsługa błędu */ }
fgets(bufor, 100, f);
fclose(f);
```

Zawsze sprawdzaj otwarcie

Pliku może nie być, może być zajęty albo bez uprawnień

Zawsze zamykaj

Niezamknięty plik może stracić dane z bufora

Dostęp sekwencyjny i swobodny

Dwa sposoby poruszania się po pliku

Sekwencyjny

Czytamy od początku do końca, linia po linii. Prosty, naturalny dla plików tekstowych.

Swobodny

Przeskakujemy do wskazanej pozycji w pliku. Wymaga rekordów o stałym rozmiarze.

```
f.seek(0)          # skok na poczatek pliku
f.seek(240)         # skok na bajt numer 240
f.tell()           # biezaca pozycja w pliku

pozycja rekordu nr i = i * rozmiar rekordu
```

Ten sam wzór co przy tablicy - dlatego dostęp swobodny działa tylko dla rekordów równej długości.

Typ wskaźnikowy

Zmienna, która przechowuje adres

```
int liczba = 42;  
int *wsk = &liczba;    // wsk przechowuje ADRES zmiennej liczba
```

zmienna	wartosc	adres
liczba	42	1000
wsk	1000	2000

```
printf("%d", *wsk);    // 42 - odczyt spod adresu  
*wsk = 99;            // zmienia wartosc zmiennej liczba
```

Operator &

Pobiera adres zmiennej

Operator *

Sięga po wartość spod adresu

Po co wskaźniki

Cztery powody

Przekazywanie dużych danych

Funkcja dostaje adres zamiast kopii całej struktury - szybciej i oszczędniej

Zmiana danych w funkcji

Funkcja może zmodyfikować zmienną wywołującego, a nie tylko swoją kopię

Pamięć przydzielana w trakcie

Tablica o rozmiarze znanym dopiero podczas działania programu

Struktury dowiązaniowe

Listy, drzewa i grafy opierają się na wskazaniach między elementami

Temat 5 pokaże, że bez wskaźników nie da się zbudować listy wiązanej - każdy element musi wiedzieć, gdzie leży następny.

Stos i sarta

Dwa obszary pamięci programu

Cecha	Stos (stack)	Sarta (heap)
Co tam trafia	Zmienne lokalne, parametry funkcji	Dane przydzielane w trakcie działania
Kto zarządza	Automatycznie - język	Programista albo odśmiecacz pamięci
Czas życia	Do końca funkcji	Do jawnego zwolnienia
Rozmiar	Ograniczony	Duży, ograniczony pamięcią systemu

```
int *tab = malloc(n * sizeof(int)); // rezerwacja na sterpie
free(tab);                          // zwolnienie - obowiazkowe w C
```

Python, Java i C# zwalniają pamięć samodzielnie - w C i C++ odpowiada za to programista.

Błędy związane ze wskaźnikami

Trzy klasyczne pułapki

Wskaźnik pusty

Odwołanie do wskaźnika o wartości pustej kończy program. Zawsze sprawdzaj przed użyciem.

Wskaźnik wiszący

Wskazuje obszar już zwolniony. Dane w tym miejscu bywają przypadkowe.

Wyciek pamięci

Program rezerwuje pamięć i nigdy jej nie zwalnia. Zużycie rośnie aż do zatrzymania.

```
int *w = malloc(100);
free(w);
*w = 5;           // wskaźnik wiszący - obszar już nie należy do nas
w = NULL;        // dobry nawyk: po zwolnieniu wyzeruj wskaźnik
```

Temat 4 – podsumowanie

Co trzeba umieć

- Typ plikowy daje dostęp do danych trwałych, przechowywanych poza pamięcią operacyjną
- Plik tekstowy zawiera znaki czytelne dla człowieka, binarny - bajty w postaci wewnętrznej
- Kolejność operacji na pliku: otwórz, czytaj lub zapisuj, zamknij
- Tryb zapisu kasuje zawartość pliku - do dopisywania służy osobny tryb
- Dostęp swobodny wymaga rekordów o stałym rozmiarze; pozycja = numer razy rozmiar
- Wskaźnik przechowuje adres; operator & pobiera adres, operator * sięga po wartość
- Zmienne lokalne leżą na stosie, dane przydzielane w trakcie działania - na sterpie
- Typowe błędy: wskaźnik pusty, wskaźnik wiszący, wyciek pamięci

5

Kolekcje: listy, kolejki, stosy, wektory

Znaczenie iteratora

lista wiązana · LIFO i FIFO · wektor · iterator

Czym jest kolekcja

Pojemnik na wiele elementów

```
kolekcja = struktura przechowująca wiele elementów  
wraz z operacjami na nich
```

```
dodaj    usun    sprawdź czy zawiera    policz    przejrzyj
```

```
roznia się tym, JAK przechowują dane i CO robią szybko
```

Wybór kolekcji to decyzja projektowa. Każda robi coś szybko kosztem czegoś innego. Dobór pod najczęstszą operację w programie decyduje o wydajności całej aplikacji.

Biblioteki standardowe języków dostarczają gotowe kolekcje - nie trzeba ich pisać samodzielnie.

Lista wiązana

Elementy połączone wskazaniem

```
wezel:  [ dane | nastepny ]
```

```
glowa
```

```
|
```

```
v
```

```
[ 10 | * ] -> [ 20 | * ] -> [ 30 | NULL ]
```

```
struct Wezel { int dane; struct Wezel *nastepny; };
```

Jednokierunkowa

Każdy węzeł zna tylko następny - idziemy wyłącznie do przodu

Dwukierunkowa

Węzeł zna następny i poprzedni - można cofać, kosztem pamięci

Lista a tablica

Każda szybka w czym innym

Operacja	Tablica / wektor	Lista wiązana
Dostęp do elementu nr i	$O(1)$ - wzór na adres	$O(n)$ - trzeba przejść od początku
Wstawienie na początek	$O(n)$ - przesuwanie	$O(1)$ - przepięcie wskazania
Wstawienie na koniec	$O(1)$ zamortyzowane	$O(1)$ przy wskaźniku na ogon
Usunięcie znanego elementu	$O(n)$ - zasypanie luki	$O(1)$ - przepięcie sąsiadów
Zużycie pamięci	Mniejsze	Większe - wskaźniki przy każdym elemencie

Reguła praktyczna: dużo odczytów po indeksie prowadzi do tablicy, dużo wstawiania i usuwania w środku - do listy.

Stos

Ostatni wszedł, pierwszy wyszedł

```
push(40)          pop() -> 40
  |                ^
  v                |
[ 40 ] <- wierzcholek
[ 30 ]
[ 20 ]
[ 10 ] <- dno
```

operacje: push - dodaj pop - zdejmij peek - podejrzuj

Cofnij operację

Każda zmiana trafia na stos

Wywołania funkcji

Stos wywołań programu

Sprawdzanie nawiasów

Otwierające na stos, zamykające
zdejmują

Kolejka

Pierwszy wszedł, pierwszy wyszedł

wejscie
(enqueue)

|

v

[40] [30] [20] [10]

koniec

początek

wyjscie
(dequeue)

^

|

operacje: enqueue - dodaj na koncu dequeue - zdejmij z początku

Kolejka zadań

Zlecenia wydruku, zadania do przetworzenia

Bufor danych

Odbiór danych szybciej, niż program je przetwarza

Przeszukiwanie wszerek

Odwiedzanie grafu poziomami

Odmiany kolejki

Dwa warianty, które warto znać

Kolejka priorytetowa

Pierwszy wychodzi element o najwyższym priorytecie, nie najstarszy. Zastosowanie: szpitalny oddział ratunkowy, zadania systemu operacyjnego.

Kolejka dwustronna

Dodawanie i usuwanie możliwe z obu końców. Zastępuje jednocześnie stos i kolejkę. W Pythonie: `collections.deque`.

```
from collections import deque

k = deque()
k.append(1)           # dodaj na koncu
k.appendleft(0)       # dodaj na początku
k.pop()               # zdejmij z konca
k.popleft()           # zdejmij z początku
```

Wektor

Tablica dynamiczna jako kolekcja

```
#include <vector>

std::vector<int> liczby;
liczby.push_back(10);           // dodanie na koncu
liczby[0];                      // dostep po indeksie - O(1)
liczby.size();                  // liczba elementow
liczby.insert(liczby.begin(), 5); // wstawienie na poczatek - O(n)
```

Wektor łączy zalety tablicy i kolekcji: ma stały dostęp po indeksie jak tablica, a jednocześnie rośnie w miarę potrzeb. To domyślny wybór, gdy nie ma powodu do innego.

Odpowiedniki: list w Pythonie, ArrayList w Javie, List<T> w C#, Array w JavaScriptcie.

Przegląd kolekcji w językach

Jak się nazywają

Struktura	Python	Java	C++
Tablica dynamiczna	<code>list</code>	<code>ArrayList</code>	<code>std::vector</code>
Lista wiązana	<code>collections.deque</code>	<code>LinkedList</code>	<code>std::list</code>
Stos	<code>list</code> (<code>append/pop</code>)	<code>Deque</code> / <code>Stack</code>	<code>std::stack</code>
Kolejka	<code>collections.deque</code>	<code>Queue</code>	<code>std::queue</code>
Słownik	<code>dict</code>	<code>HashMap</code>	<code>std::map</code>
Zbiór	<code>set</code>	<code>HashSet</code>	<code>std::set</code>

Zbiór przechowuje elementy bez powtórzeń i bez ustalonej kolejności - przydaje się do usuwania duplikatów.

Iterator

Jednolity sposób przeglądania kolekcji

```
problem: kazda kolekcja jest zbudowana inaczej
  tablica  -> indeksy      lista -> wskazania
  slownik  -> kubelki      zbior  -> ?
```

```
iterator ukrywa te roznice za dwiema operacjami:
  czy jest nastepny element?      podaj nastepny element
```

```
for (String imie : lista)      // Java - for each
for element in kolekcja:      # Python - ten sam zapis dla listy,
                              #          slownika i zbioru
```

Pętla for each działa dzięki iteratorowi - język sam go tworzy w tle.

Iterator w praktyce

Dlaczego to wygodne i gdzie uważać

Niezależność od struktury

Zmiana listy na zbiór nie wymusza zmiany pętli przeglądającej dane

Przetwarzanie leniwe

Elementy można wyliczać na bieżąco, bez trzymania wszystkiego w pamięci

Jeden przebieg

Zużyty iterator trzeba utworzyć od nowa, żeby przejść kolekcję ponownie

Modyfikacja w trakcie

Dodawanie lub usuwanie elementów podczas pętli unieważnia iterator

```
# ZLE - usuwanie podczas iteracji
for x in lista:
    if warunek(x): lista.remove(x)
# DOBRZE - budujemy nowa liste
lista = [x for x in lista if not warunek(x)]
```

Temat 5 – podsumowanie

Co trzeba umieć

- Kolekcja przechowuje wiele elementów wraz z operacjami na nich
- Lista wiązana składa się z węzłów: dane plus wskazanie na następny element
- Tablica daje szybki dostęp po indeksie, lista - szybkie wstawianie i usuwanie
- Stos działa według zasady LIFO: push, pop, peek
- Kolejka działa według zasady FIFO: enqueue na końcu, dequeue z początku
- Wektor to tablica dynamiczna - stały dostęp po indeksie i rosnący rozmiar
- Iterator daje jednolity sposób przeglądania dowolnej kolekcji
- Modyfikowanie kolekcji podczas iteracji unieważnia iterator

6

Projektowanie zestawu danych

Dobór struktury do problemu programistycznego

analiza · schemat decyzyjny · przykład · ocena

Na czym polega projektowanie danych

Decyzja przed napisaniem kodu

1. Jakie dane program przechowuje?
2. Jakie operacje wykonuje na nich najczesciej?
3. Ile danych bedzie w najgorszym przypadku?
4. Jaka struktura wykonuje te operacje najszybciej?
5. Czy dane maja przetrwac zamkniecie programu?

Kolejność ma znaczenie. Struktura danych wynika z operacji, a nie odwrotnie. Dobrze dobrana struktura sprawia, że algorytm staje się prosty; źle dobrana wymusza obchodzenie problemów w kodzie.

Zmiana struktury danych w gotowej aplikacji pociąga za sobą przepisanie dużej części kodu.

Schemat decyzyjny

Cztery pytania prowadzące do struktury

1 Czy elementy są tego samego typu?

Nie - potrzebny rekord. Tak - idź dalej

2 Czy kolejność ma znaczenie?

Tak - tablica, lista, stos lub kolejka. Nie - zbiór lub słownik

3 Jak szukam elementu?

Po pozycji - tablica. Po nazwie lub identyfikatorze - słownik

4 Gdzie dodaję i usuwam?

Na końcu - wektor. Na początku lub w środku - lista. Jeden koniec - stos lub kolejka

Przykład: dziennik ocen

Od wymagań do struktury

WYMAGANIA

- każdy uczeń ma imię, nazwisko, numer w dzienniku
- uczeń ma wiele ocen z wielu przedmiotów
- często: wyświetl oceny ucznia, policz średnią z przedmiotu
- rzadko: dopisz ucznia w ciągu roku

ROZWIĄZANIE

```
rekord Uczeń { imię, nazwisko, numer, oceny }  
oceny: słownik   przedmiot -> lista ocen  
klasa: wektor rekordów Uczeń
```

Słownik, bo oceny wyszukujemy po nazwie przedmiotu. Lista wewnątrz, bo kolejność wystawiania ocen ma znaczenie.

Przykład: historia przeglądarki

Ta sama metoda, inny wynik

WYMAGANIA

- zapamiętaj odwiedzone adresy
- przycisk wstecz cofa do poprzedniej strony
- przycisk dalej wraca do strony opuszczonej przyciskiem wstecz

ROZWIĄZANIE

dwa stosy: wstecz oraz dalej

nowa strona -> push na stos wstecz, czyszczenie stosu dalej

przycisk wstecz -> pop ze stosu wstecz, push na stos dalej

Operacja zawsze dotyczy ostatniego elementu, więc naturalnym wyborem jest stos.

Typowe błędy w doborze struktury

Czego unikać

Tablice równoległe

Osobna tablica na każde pole zamiast tablicy rekordów - dane rozjeżdżają się przy pierwszej modyfikacji

Lista zamiast słownika

Wyszukiwanie po identyfikatorze przez przeglądanie całej listy - zbędne spowolnienie

Stały rozmiar na wszelki wypadek

Tablica na tysiąc elementów, gdy zwykle jest ich dziesięć - i tak zabraknie w najgorszym przypadku

Struktura dobrana po napisaniu kodu

Dopasowywanie danych do gotowych funkcji zamiast odwrotnie

Pytanie sprawdzające projekt: ile linii kodu trzeba będzie zmienić, gdy dojdzie nowe pole albo gdy danych będzie sto razy więcej?

Jak ocenię Wasz projekt danych

Kryteria z ćwiczeń

Kryterium	Na czym polega
Kompletność	Wszystkie dane z treści zadania mają swoje miejsce w projekcie
Trafność struktury	Wybór pasuje do najczęstszej operacji opisanej w wymaganiach
Uzasadnienie	Każdy wybór opatrzony jednym zdaniem wyjaśnienia
Typy pól	Pola rekordów mają dobrane typy proste z działu I
Odporność na zmianę	Dodanie nowego pola nie wymaga przebudowy całości

Ćwiczenia do tej lekcji znajdują się w pliku 03-struktury-danych-cwiczenia.md

Ściąga – całość działu

Którą strukturę wybrać

Potrzeba	Struktura
Dostęp po pozycji, rozmiar znany z góry	Tablica statyczna
Dostęp po pozycji, rozmiar rośnie	Tablica dynamiczna / wektor
Siatka wierszy i kolumn	Tablica dwuwymiarowa
Wyszukiwanie po nazwie lub identyfikatorze	Tablica asocjacyjna / słownik
Opis obiektu o wielu różnych cechach	Rekord / struktura
Częste wstawianie i usuwanie w środku	Lista wiązana
Dostęp tylko do ostatniego elementu	Stos (LIFO)
Obsługa w kolejności zgłoszeń	Kolejka (FIFO)
Dane mają przetrwać zamknięcie programu	Plik lub baza danych

tablica rekord lista stos słownik

Dziękuję za uwagę!

Ćwiczenia do działu: 03-struktury-danych-cwiczenia.md

Opracował: Bartosz Bryniarski